MIPS Instruction Set

---

1. Introduction to MIPS

- MIPS (Microprocessor without Interlocked Pipeline Stages) is a RISC (Reduced Instruction Set Computer) architecture.
- Commercialized by **MIPS Technologies**, widely used in embedded systems (network/storage equipment, cameras, printers).
- Simple, fixed-size instructions enable efficient pipelining and performance.
- Instruction set commonly used in textbooks for teaching.

2. Register Operands

MIPS uses 32 general-purpose registers (32-bit wide) for faster operations than memory.

- `$zero` – Constant 0 (Register 0)
- `$t0 { $t9` – Temporaries (Registers 8–15, 24–25)
- `$s0 { $s7` – Saved values (Registers 16–23)
- `$a0 { $a3` – Arguments (Registers 4–7)
- `$v0, $v1` – Return values (Registers 2–3)
- `$sp` – Stack pointer (Register 29)
- `$ra` – Return address (Register 31)

2.1 Example: Register Usage

```
add $t0, $s1, $s2   # t0 = g + h
add $t1, $s3, $s4   # t1 = i + j
sub $s0, $t0, $t1   # f = t0 - t1
```

2.2 Special Case: $zero

```
add $t2, $s1, $zero   # Copies $s1 to $t2 (acts like "move")
```

3. Arithmetic Instructions

3.1 Core Instructions

Format

```
add $d, $s, $t          # $d = $s + $t
sub $d, $s, $t          # $d = $s - $t
mult $s, $t             # HI/LO = $s * $t
div $s, $t              # LO = quotient, HI = remainder
```

3.2 Example: C to MIPS Translation

```
C: f = (g + h) - (i + j);
MIPS:
add $t0, $s1, $s2   # t0 = g + h
add $t1, $s3, $s4   # t1 = i + j
sub $s0, $t0, $t1   # f = t0 - t1
```

4. Logical Operations

4.1 Bitwise Instructions

Instruction	Operation
`and $d, $s, $t`	Bitwise AND
`or $d, $s, $t`	Bitwise OR
`nor $d, $s, $t`	Bitwise NOR (NOT OR)
`xor $d, $s, $t`	Bitwise XOR (pseudo)

4.2 Practical Examples

- **Masking with AND**:

```
and $t0, $t1, $t2   # Keeps bits where $t2=1
```

- **Setting Bits with OR**:

```
or $t0, $t1, $t2    # Sets bits to 1 where $t2=1
```

- **Inversion with NOR**:

```
nor $t0, $t1, $zero # $t0 = NOT $t1
```

5. Shift Operations

5.1 Instructions

Instruction	Purpose
`sll $d, $t, shamt`	Shift left (multiply by $2^{\texttt{shamt}}$)
`srl $d, $t, shamt`	Shift right (unsigned division)
`sra $d, $t, shamt`	Arithmetic shift right (preserves sign)

5.2 Examples

```
sll $t0, $s0, 3   # $t0 = $s0 * 8
sra $t0, $s0, 2   # $t0 = $s0 / 4 (signed)
```

6. Immediate Operands

6.1 Common Instructions

Immediate Instructions

```
addi $t0, $t1, 10                    # $t0 = $t1 + 10
andi $t0, $t1, 0xFF                  # Bitwise AND with constant
slti $t0, $t1, 20                    # Set if $t1 ¡ 20 (signed)
```

6.2 Example

```
addi $t0, $s1, 2    # $t0 = g + 2
addi $t1, $s2, -3   # $t1 = h - 3
sub $s0, $t0, $t1   # f = (g+2) - (h-3)
```

7. 32-bit Constants with `lui` and `ori`

- `lui rt, constant` loads upper 16 bits.
- Combine with `ori` for full 32-bit values.

```
lui $s0, 61        # $s0 = 0x3D0000
ori $s0, $s0, 2304 # $s0 = 0x3D0900
```

Note

MIPS immediate fields are 16-bit. For 32-bit constants, construct upper/lower halves separately.

8. Summary Table

Category	Key Instructions	Purpose
Arithmetic	`add, sub, mult, div`	Integer math
Logical	`and, or, nor, xor`	Bit manipulation
Shift	`sll, srl, sra`	Bit shifting
Immediate	`addi, andi, ori, slti`	Constants in ops
Data Movement	`lui, ori`	32-bit constant setup